

DEV_IMPL_GUIDE - DD: BIL-01-PAY-01 — Payment Application

1. Purpose

This guide explains how to implement DD_BIL-01-PAY-01-Payment_Application-v1.0.md as executable SOPHIX behavior.

Use it with:

- DD_BIL-01-PAY-01-Payment_Application-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md when this DD participates in subscription workflow operations.
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md when this DD references worker topics.

2. Runtime Actors

Runtime component	Responsibility
API pod / module service	Receives requests, validates, loads authoritative state, starts commands.
Owning module database	Stores tables/configuration owned by this DD.
Reference modules	Provide data through approved APIs/client wrappers.
Camunda	Runs BPMN process when this DD is workflow-driven.
Worker apps	Execute logical worker topics.
Drools	Evaluates packages when rules are declared.
Kafka/outbox	Publishes success/failure and integration events.

3. Before Implementation

Confirm these contracts from the DD:

Contract	Value
Operational anchor	account_id; customer_id is denormalized identity/search context.
Endpoint scan reference	GET /api/credit-balance/{accountId}
BPMN/process key	None for normal receipt/allocation; admin mark-paid-offline may be called by billing workflow screens.
Drools package	Not required for v1; allocation policy is config-driven.
Tables	payment_ledger, payment_invoice_allocation, account_credit_balance, pay_01_config.
Events	InvoiceIssued, PaymentApplied, PaymentReceivedUnallocated, PaymentReversed

4. Execution Flow

1. Apply migrations and seed configuration/catalog rows.
2. Implement the module service/repository layer around the tables and invariants.
3. Implement public/internal APIs from the DD.
4. Implement rule evaluation if the DD defines Drools packages.
5. Implement event production/consumption through outbox where the DD defines events.
6. Add integration tests for table constraints, API validation, and event payloads.

5. API And Command Handling

Implement these endpoints first:

- GET /api/credit-balance/{accountId}
- GET /api/payments/{id}
- GET /api/payments/{paymentId}
- GET /api/payments?accountId=acc_01HX_HOME&from=2026-05-01&to=2026-05-31&limit=20
- GET /api/payments?customerId=cust_01HX...&from=2026-05-01&to=2026-05-31&limit=20
- POST /api/admin/credit-balance/{accountId}/adjust
- POST /api/admin/credit-balance/{accountId}/apply
- POST /api/admin/payments/pmt_01HZ...C/allocate-surplus
- POST /api/admin/payments/{id}/allocate-surplus
- POST /api/payments/pmt_01HZ...D/reverse
- POST /api/payments/receive
- POST /api/payments/{paymentId}/reverse
- POST /api/wallets/credit
- POST /api/wallets/debit

Implementation rules:

- Resolve actor user and role from auth context, not from public request body.
- Check idempotency before inserting a new operation or creating a side effect.
- Return the existing operation/result for the same idempotency key and same request hash.
- Return 409 IDEMPOTENCY_KEY_REUSED for the same key with a different request hash.
- Use transactions or unique constraints for concurrency-sensitive creation.

5.1 Flow - Receive A Gateway Payment

The gateway adapter or admin tool calls PAY-01:

```
POST /api/payments/receive
Authorization: Bearer <payment-gateway-or-admin-token>
Idempotency-Key: pay:acc_01HX_HOME:mpesa_NK4XX7QP9
Content-Type: application/json
```

```
{
  "operatorCode": "WIK",
  "customerId": "cust_01HX",
  "accountId": "acc_01HX_HOME",
  "paidAmount": 6402.50,
  "currency": "KES",
  "paymentMethod": "MPESA",
  "paymentReference": "mpesa_NK4XX7QP9",
  "paidAt": "2026-05-18T10:30:00Z",
  "targetInvoiceId": null,
  "targetWalletRef": null
}
```

PAY-01 executes:

1. Validate accountId belongs to customerId and operatorCode through ILM-CFG-01 client/cache snapshot.
2. Reserve idempotency on (account_id, payment_reference); return the previous result for same hash, return 409 for different hash.
3. Resolve billing context:
 - if targetInvoiceId exists, load invoice and require invoice.account_id = account_id;
 - if targetWalletRef exists, validate BIL-05 wallet belongs to the same account;
 - if neither exists, load open invoices for the account and determine whether auto-allocation is allowed.
4. Insert payment_ledger.
5. Apply allocation:

- POSTPAID invoice path: create `payment_invoice_allocation` rows for invoices under the same `account_id`;
- PREPAID wallet path: call BIL-05 with `account_id`, `customer_id`, `amount`, `currency`, and idempotency key.
- 6. If surplus remains, write `account_credit_balance` or hold for manual review according to `pay_01_config`.
- 7. Emit `PaymentApplied` once per invoice allocation, or `PaymentReceivedUnallocated` when money is recorded but not allocated.

Never allocate a payment across two accounts owned by the same customer unless a future explicit transfer workflow is approved.

6. Data Loading

Load data in this order:

1. Authoritative aggregate state from the owning module.
2. Operator/module config from this DD's config tables.
3. Catalog/reference data from approved APIs or client wrappers.
4. Current in-flight operation state from the framework, where relevant.

Do not read current mutable state from cache for state-changing operations.

7. Drools Configuration

Packages found:

- No Drools package explicitly declared.

For each package:

- Define fact classes that mirror the request, current state, config, and reference data.
- Keep rule output as explicit decision fields.
- Unit-test accept and reject cases from the DD.
- BPMN should re-check critical rules after start if state may have changed.

8. BPMN And Worker Topics

Worker topics found:

- `billing-charging`
- `mark-paid-offline`

Implementation rules:

- Model service tasks with `camunda:type="external"` and the exact topic string.
- Worker handler must validate required variables before calling downstream services.
- Store outputs as process variables with stable names.
- Use message correlation keys exactly as specified by the DD.
- Add compensation paths for each external side effect.

9. Events

Events found:

- `InvoiceIssued`
- `PaymentApplied`
- `PaymentReceivedUnallocated`
- `PaymentReversed`

Event rules:

- Emit success only after authoritative commit.
- Emit rejected/failure event only after compensation/final failure decision.
- Use the outbox table/dispatcher; do not publish directly inside the transaction boundary unless the platform standard allows it.

10. Smoke Tests

Add tests for:

- Happy path.
- Invalid role or trigger.
- Invalid state.
- Missing config/catalog row.
- Dependency failure.
- Retry/idempotency.
- Event payload correctness.
- Worker topic failure and compensation.

11. Common Mistakes

- Treating worker-topic names as shell commands.
- Querying another module's database directly.
- Reading current state from cache in a state-changing flow.
- Emitting success before final commit.
- Hiding manual recovery paths for partial external side effects.
- Creating one deployment per topic when a consolidated worker app would be easier to operate.